

Tuples – Creating tuples, Accessing elements, Understanding immutability, Slicing, Unpacking, and Iterating over tuples.

टपल (Tuples) – टपल बनाना, तत्वों को एक्सेस करना, अपरिवर्तनीयता (Immutability) को समझना, स्लाइसिंग, अनपैकिंग और टपल पर इटरेट करना।

टपल (tuple) items का एक क्रमबद्ध (ordered), अपरिवर्तनीय (immutable) संग्रह है। टपल कोष्ठक () का उपयोग करके लिखे जाते हैं।

1. Creating Tuples 1. टपल बनाना

हम टपल कई तरीकों से बना सकते हैं।

- कोष्ठक (parentheses) का उपयोग करके

```
numbers = (10, 20, 30, 40)
```

- बिना कोष्ठक (parentheses) के

```
fruits = 'apple', 'banana', 'mango'
```

- एकल-तत्व टपल (कॉमा अनिवार्य है)

```
one = (5,)
```

- खाली टपल

```
empty = ()
```

उदाहरण :

```
numbers = (10, 20, 30, 40)
fruits = 'apple', 'banana', 'mango'
one = (5,)
empty = ()
```

Output / Type आउटपुट / प्रकार

```
(10, 20, 30, 40)      <class 'tuple'>
('apple', 'banana', 'mango') <class 'tuple'>
(5,)                  <class 'tuple'>
()                    <class 'tuple'>
```

2. Accessing Elements 2. तत्वों को एक्सेस करना

टपल के तत्वों (elements) को इंडेक्स (index) की सहायता से एक्सेस किया जाता है। इंडेक्स (Index) 0 से शुरू होता है।

उदाहरण :

```
t = ('p', 'y', 't', 'h', 'o', 'n')
```

इंडेक्स (Index)	0	1	2	3	4	5
मान (Value)	'p'	'y'	't'	'h'	'o'	'n'

एक्सेस करना (Accessing) :

```
t[0] → 'p' # first element
t[3] → 'h' # element at index 3
t[-1] → 'n' # last element
t[-2] → 'o' # second last element
```

3. Understanding Immutability 3. अपरिवर्तनीयता (Immutability) को समझना

टपल अपरिवर्तनीय (immutable) होते हैं, अर्थात टपल बन जाने के बाद हम उसके तत्वों को बदल, जोड़ या हटा नहीं सकते।

उदाहरण :

```
t = (10, 20, 30)
t[0] = 100 # TypeError
t.append(40) # AttributeError
del t[1] # TypeError
```

नोट :

हम पूरे टपल को दोबारा असाइन (reassign) कर सकते हैं, लेकिन अलग-अलग तत्वों को बदल नहीं सकते।

उदाहरण :

```
t = (10, 20, 30)
t = (40, 50) # valid
```

4. Slicing Tuples 4. टपल की स्लाइसिंग

हम टपल को लिस्ट (list) की तरह ही स्लाइस कर सकते हैं।

उदाहरण :

```
t = (10, 20, 30, 40, 50, 60)
```

एक्सप्रेशन (Expression)	परिणाम (टपल)
t[1:4]	(20, 30, 40)
t[:3]	(10, 20, 30)
t[3:]	(40, 50, 60)
t[::2]	(10, 30, 50)
t[::-1]	(60, 50, 40, 30, 20, 10)

Note : नोट :

- स्लाइसिंग एक नया टपल लौटाती है। मूल टपल अपरिवर्तित रहता है।

5. Unpacking Tuples 5. टपल की अनपैकिंग

हम एक टपल को कई वैरिएबल्स (variables) में अनपैक कर सकते हैं।

उदाहरण :

```
t = ('a', 'b', 'c')
x, y, z = t
print(x, y, z) # Output: a b c
```

Unpacking with * (for extra values)
* के साथ अनपैकिंग (अतिरिक्त मानों के लिए)

```
t = (1, 2, 3, 4, 5)
a, b, *rest = t
print(a) # 1
print(b) # 2
print(rest) # [3, 4, 5]
```

Swapping using unpacking अनपैकिंग द्वारा स्विपिंग (मानों की अदला-बदली)

```
a, b = 10, 20
a, b = b, a
print(a, b) # Output: 20 10
```

6. Iterating Over Tuples 6. टपल पर इटरेट करना

हम लूप (loops) का उपयोग करके टपल पर इटरेट कर सकते हैं।

उदाहरण :

```
t = ('red', 'green', 'blue')
for color in t:
    print(color)
```

Output : आउटपुट :

- red
- green
- blue

Summary सारांश

- टपल क्रमबद्ध (ordered) और अपरिवर्तनीय (immutable) होते हैं।
- हम इंडेक्स (index) की सहायता से तत्वों को एक्सेस करते हैं।
- स्लाइसिंग (slicing) एक नया टपल लौटाती है।
- हम टपल के मानों को वैरिएबल्स (variables) में अनपैक कर सकते हैं।
- हम लूप (loops) का उपयोग करके टपल पर इटरेट कर सकते हैं।



Tuples – Creating tuples.

टपल (Tuples) – टपल बनाना।

टपल (tuple) कोष्ठक () के अंदर लिखे गए items का एक क्रमबद्ध (ordered), अपरिवर्तनीय (immutable) संग्रह है।

1. Creating Tuples 1. टपल बनाना

हम टपल कई तरीकों से बना सकते हैं।

(a) Using parentheses (most common way)

(a) कोष्ठक (parentheses) का उपयोग करके (सबसे आम तरीका)

```
t = (10, 20, 30, 40) # creates a tuple with 4 elements
```

(b) Without parentheses

(b) बिना कोष्ठक (parentheses) के

```
t = 10, 20, 30, 40 # parentheses are optional
```

(c) Single element tuple

(c) एकल-तत्व टपल

```
t = (5,) # comma is mandatory for single element tuple
```

(d) Empty tuple

(d) खाली टपल

```
t = () # empty tuple
```

Key Points मुख्य बिंदु

- टपल के तत्व कोष्ठक () के अंदर लिखे जाते हैं।
- टपल क्रमबद्ध (ordered) होते हैं।
- टपल अपरिवर्तनीय (immutable) होते हैं (बदले नहीं जा सकते)।
- डुप्लिकेट मान (duplicate values) की अनुमति है।

2. Examples 2. उदाहरण

कोड (Code)	बना हुआ टपल (Tuple Created)	प्रकार (Type)
t1 = (1, 2, 3, 4)	(1, 2, 3, 4)	<class 'tuple'>
t2 = 'a', 'b', 'c'	('a', 'b', 'c')	<class 'tuple'>
t3 = (100,)	(100,)	<class 'tuple'>
t4 = ()	()	<class 'tuple'>

3. Example Program 3. उदाहरण प्रोग्राम

```
# Creating tuples
t1 = (10, 20, 30)
t2 = 'apple', 'banana', 'mango'
t3 = (5,) # single element tuple
t4 = () # empty tuple
print(t1, type(t1))
print(t2, type(t2))
print(t3, type(t3))
print(t4, type(t4))
```

Output आउटपुट

```
(10, 20, 30) <class 'tuple'>
('apple', 'banana', 'mango') <class 'tuple'>
(5,) <class 'tuple'>
() <class 'tuple'>
```

Tuples – Accessing Elements

टपल (Tuples) – तत्वों को एक्सेस करना

टपल के तत्व (elements) इंडेक्स (index) की सहायता से एक्सेस किए जाते हैं।

Python में इंडेक्सिंग (indexing) 0 से शुरू होती है।

1. Accessing Elements 1. तत्वों को एक्सेस करना

उदाहरण :

```
t = ('red', 'green', 'blue', 'yellow')
```

इंडेक्स (Index)	0	1	2	3
मान (Value)	'red'	'green'	'blue'	'yellow'

एक्सेस करना (Accessing) :

```
t[0] → 'red'      # first element
t[1] → 'green'    # element at index 1
t[2] → 'blue'     # element at index 2
t[3] → 'yellow'   # last element
```

2. Negative Indexing 2. ऋणात्मक इंडेक्सिंग (Negative Indexing)

हम ऋणात्मक इंडेक्स (negative index) का उपयोग करके भी तत्वों को एक्सेस कर सकते हैं।

इंडेक्स (Index)	-4	-3	-2	-1
मान (Value)	'red'	'green'	'blue'	'yellow'

ऋणात्मक इंडेक्स से एक्सेस करना (Accessing with Negative Index) :

```
t[-4] → 'red'
t[-3] → 'green'
t[-2] → 'blue'
t[-1] → 'yellow'
```

3. Example Program 3. उदाहरण प्रोग्राम

```
# Accessing elements of a tuple
t = ('red', 'green', 'blue', 'yellow')
print(t[0])    # red
print(t[1])    # green
print(t[2])    # blue
print(t[3])    # yellow
print(t[-1])   # yellow
print(t[-3])   # green
```

Output : आउटपुट :

```
red
green
blue
yellow
yellow
green
```

Key Points मुख्य बिंदु

- टपल के तत्वों को एक्सेस करने के लिए इंडेक्स (index) का उपयोग करें।
- धनात्मक इंडेक्स (positive index) 0 से शुरू होता है।
- ऋणात्मक इंडेक्स (negative index) -n से -1 तक शुरू होता है।

Tuples – understanding immutability

टपल (Tuples) – अपरिवर्तनीयता (Immutability) को समझना

टपल अपरिवर्तनीय (immutable) होते हैं, अर्थात एक बार टपल बन जाने के बाद उसके तत्वों को बदला, जोड़ा या हटाया नहीं जा सकता।

मुख्य विचार (Key idea) :

आप टपल के तत्वों को एक्सेस कर सकते हैं, लेकिन बनने के बाद उन्हें बदल नहीं सकते।

Example 1 : Valid operations (accessing) उदाहरण 1 :

मान्य ऑपरेशन (एक्सेस करना)

```
t = (10, 20, 30, 40)
print(t[0])      # 10
print(t[2])      # 30
print(t[-1])     # 40
print(len(t))    # 4
```

← तत्वों को
एक्सेस करना
अनुमत
(allowed) है।

Example 2 : Invalid operations (modifying) उदाहरण

2 : अमान्य ऑपरेशन (बदलाव करना)

```
t = (10, 20, 30, 40)
t[1] = 200      #
Error
TypeError: 'tuple' object
does not support item
assignment

t.append(50)     #
Error
AttributeError: 'tuple'
object has no attribute
'append'

del t[2]         #
Error
TypeError: 'tuple' object
doesn't support item
deletion

t[1:3] = (200, 300) #
Error
TypeError: 'tuple' object
does not support item
assignment
```

← टपल में बदलाव
करने का कोई भी
प्रयास एरर
(error) उत्पन्न
करता है।

Example 3 : Creating a new tuple (allowed) उदाहरण

3 : नया टपल बनाना (अनुमत)

```
t = (10, 20, 30, 40)
t2 = t + (50,)      #
concatenation
t3 = t[:2] + (100,) + t[2:] # slicing
print(t2)           # (10, 20, 30, 40, 50)
print(t3)           # (10, 20, 100, 30, 40)
```

बदलाव करने के बजाय, हम एक नया टपल बनाते हैं।

Key Points मुख्य बिंदु

- टपल अपरिवर्तनीय (immutable) होते हैं (बदले नहीं जा सकते)।
- तत्वों को असाइन (assign), डिलीट (delete) या संशोधित (modify) नहीं किया जा सकता।
- कंकेटिनेशन (concatenation) या स्लाइसिंग (slicing) से नए टपल बनाए जा सकते हैं।
- अपरिवर्तनीयता (Immutability) टपल को सुरक्षित और तेज़ बनाती है।

Summary सारांश

- लिस्ट (Lists) परिवर्तनशील (mutable) होती हैं (बदली जा सकती हैं), लेकिन टपल अपरिवर्तनीय (immutable) होते हैं (बदले नहीं जा सकते)।
- जब डेटा स्थिर (constant) रहना चाहिए, तब टपल का उपयोग करें।

Tuples – Slicing

टपल (Tuples) – स्लाइसिंग

स्लाइसिंग (Slicing) हमें टपल का एक हिस्सा निकालने की सुविधा देती है। निकाला गया हिस्सा एक नए टपल के रूप में लौटाया जाता है।

1. Syntax 1. सिंटैक्स

`t[start : stop : step]`

जहाँ,

- start → शुरुआती इंडेक्स (डिफ़ॉल्ट 0 है)
- stop → अंतिम इंडेक्स (शामिल नहीं होता)
- step → स्टेप का आकार (डिफ़ॉल्ट 1 है)

2. Example Tuple 2. उदाहरण टपल

आइए एक टपल `t = (10, 20, 30, 40, 50, 60)` पर विचार करें।

इंडेक्स (Index)	0	1	2	3	4	5
मान (Value)	10	20	30	40	50	60

3. Slicing Examples 3. स्लाइसिंग के उदाहरण

एक्सप्रेशन (Expression)	परिणाम (टपल)	स्पष्टीकरण (Explanation)
<code>t[1:4]</code>	<code>(20, 30, 40)</code>	इंडेक्स 1 से 3 तक के तत्व
<code>t[:4]</code>	<code>(10, 20, 30, 40)</code>	शुरुआत से इंडेक्स 3 तक
<code>t[2:]</code>	<code>(30, 40, 50, 60)</code>	इंडेक्स 2 से अंत तक
<code>t[:]</code>	<code>(10, 20, 30, 40, 50, 60)</code>	पूरा टपल (एक कॉपी)
<code>t[::2]</code>	<code>(10, 30, 50)</code>	हर दूसरा तत्व
<code>t[1::2]</code>	<code>(20, 40, 60)</code>	इंडेक्स 1 से शुरू करके, हर दूसरा तत्व
<code>t[::-1]</code>	<code>(60, 50, 40, 30, 20, 10)</code>	टपल का उलटा (reverse)
<code>t[4:1:-1]</code>	<code>(50, 40, 30)</code>	इंडेक्स 4 से नीचे इंडेक्स 2 तक

उदाहरण प्रोग्राम (Example Program)

```
t = (10, 20, 30, 40, 50, 60)
print(t[1:4])    # (20, 30, 40)
print(t[:3])     # (10, 20, 30)
print(t[3:])     # (40, 50, 60)
print(t[::-1])   # (60, 50, 40, 30, 20, 10)
print(t[::2])    # (10, 30, 50)
```

मूल टपल अपरिवर्तित रहता है

`t = (10, 20, 30, 40, 50, 60)`

Key Points मुख्य बिंदु

- stop इंडेक्स शामिल नहीं होता।
- स्लाइसिंग मूल टपल को नहीं बदलती।
- स्लाइसिंग हमेशा एक नया टपल लौटाती है।

Tuples – Unpacking

टपल (Tuples) – अनपैकिंग

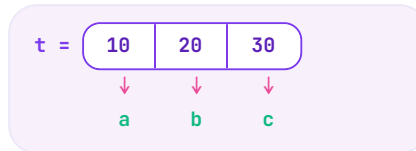
अनपैकिंग (Unpacking) हमें टपल से मान निकालकर उन्हें एक ही स्टेटमेंट में कई वेरिएबल्स (variables) को असाइन करने की सुविधा देती है।

1. Basic Unpacking 1. बेसिक अनपैकिंग

वेरिएबल्स की संख्या टपल के तत्वों की संख्या के बराबर होनी चाहिए।

उदाहरण :

```
t = (10, 20, 30)
a, b, c = t
print(a) # 10
print(b) # 20
print(c) # 30
```



Key Points मुख्य बिंदु

- अनपैकिंग कोड को साफ़ और पठनीय (readable) बनाती है।
- वेरिएबल्स की संख्या टपल के तत्वों की संख्या के बराबर होनी चाहिए।
- मानों की अदला-बदली (swapping) करने और फ़ंक्शन से कई मान लौटाने के लिए उपयोगी है।

2. Using * for Extra Elements 2. अतिरिक्त तत्वों के लिए * का उपयोग

हम बाकी बचे तत्वों को एक लिस्ट (list) में इकट्ठा करने के लिए * का उपयोग कर सकते हैं।

उदाहरण :

```
t = (10, 20, 30, 40, 50)
a, b, *rest = t
print(a) # 10
print(b) # 20
print(rest) # [30, 40, 50]
```



3. Swapping Variables 3. वेरिएबल्स की अदला-बदली (Swapping)

अनपैकिंग का उपयोग अक्सर मानों की अदला-बदली (swap) करने के लिए किया जाता है।

उदाहरण :

```
a = 10
b = 20
a, b = b, a # unpacking
print(a) # 20
print(b) # 10
```

4. Ignoring Values 4. मानों को अनदेखा करना

अवांछित मानों को अनदेखा करने के लिए _ का उपयोग करें।

उदाहरण :

```
t = (100, 200, 300)
_, b, _ = t
print(b) # 200
```

5. Function Returning Multiple Values 5. कई मान लौटाने वाला फ़ंक्शन

एक फ़ंक्शन (function) टपल लौटा सकता है, जिसे अनपैक किया जा सकता है।

उदाहरण :

```
def get_person():
    return ('Alice', 25, 'Engineer')
```



```
name, age, profession = get_person()
print(name) # Alice
print(age) # 25
print(profession) # Engineer
```

Output : आउटपुट :

Alice
25
Engineer

Tuples – iterating

टपल (Tuples) – इटरेट करना (iterating)

इटरेशन (Iteration) का अर्थ है टपल के हर तत्व को एक-एक करके एक्सेस करना।

चूँकि टपल अपरिवर्तनीय (immutable) होते हैं, हम इटरेशन के दौरान उन्हें बदल नहीं सकते, लेकिन हर तत्व को पढ़ सकते हैं।

1. Iterating using a for loop

1. for लूप द्वारा इटरेट करना

टपल के हर तत्व को एक्सेस करने के लिए हम for लूप का उपयोग कर सकते हैं।

उदाहरण :

```
t = (10, 20, 30, 40)
for item in t:
    print(item)    # Output
```

Output : आउटपुट :

```
10
20
30
40
```

2. Iterating using index and range()

2. इंडेक्स और range() द्वारा इटरेट करना

हम इंडेक्स (index) और range() का उपयोग करके भी टपल पर इटरेट कर सकते हैं।

उदाहरण :

```
t = (10, 20, 30, 40)
for i in range(len(t)):
    print(t[i])    # Output
```

Output : आउटपुट :

```
10
20
30
40
```

3. Using enumerate()

3. enumerate() का उपयोग

enumerate() की सहायता से हम इंडेक्स (index) और मान (value) दोनों प्राप्त कर सकते हैं।

उदाहरण :

```
t = ('red', 'green', 'blue')
for index, value in enumerate(t):
    print(index, '->', value)    # Output
```

Output : आउटपुट :

```
0 → red
1 → green
2 → blue
```

4. Iterating with tuple unpacking

4. टपल अनपैकिंग के साथ इटरेट करना

यदि किसी टपल में टपल हों, तो हम अनपैकिंग (unpacking) का उपयोग करके इटरेट कर सकते हैं।

उदाहरण :

```
t = ((1, 'A'), (2, 'B'), (3, 'C'))
for num, ch in t:
    print(num, ': ', ch)    # Output
```

Output : आउटपुट :

```
1 : A
2 : B
3 : C
```

Key Points मुख्य बिंदु

- टपल पर इटरेट करने के लिए हम for लूप का उपयोग करते हैं।
- इटरेशन के दौरान टपल को बदला नहीं जा सकता।
- उन्नत इटरेशन (advanced iteration) के लिए हम range(), enumerate() या अनपैकिंग (unpacking) का उपयोग कर सकते हैं।
- इटरेशन हमें टपल के हर तत्व को कुशलता से प्रोसेस करने में मदद करता है।